

EsolangIDE 外部解释器适配

cpp 案例

```
1  #include <bits/stdc++.h>
2
3  /**
4   * @brief 解释器逻辑
5   * @param tokens 代码行，每个元素都是一个代码token
6   * @param input 输入流，使用 input >> a 来代替 std::cin >> a
7   * @return 程序返回值，return 0以正常结束程序
8   */
9  int interpreter(const std::vector<std::string> &tokens, std::ifstream
&input) {
10     // TODO 在这里写解释器逻辑
11 }
12
13 int main(int argc, char *argv[]) {
14     if (argc < 3) {
15         std::cerr << "Usage: " << argv[0] << " <code_file> <input_file>" <<
std::endl;
16         return 1;
17     }
18
19     std::ifstream code_file(argv[1]);
20     if (!code_file) {
21         std::cerr << "Failed to open code file: " << argv[1] << std::endl;
22         return 1;
23     }
24
25     std::vector<std::string> tokens;
26     std::string token;
27     while (code_file >> token) {
28         tokens.push_back(std::move(token));
29     }
30     code_file.close();
31
32     std::ifstream input_file(argv[2]);
33     if (!input_file) {
34         std::cerr << "Failed to open input file: " << argv[2] << std::endl;
35         return 1;
36     }
37
38     std::ios_base::sync_with_stdio(false);
39     std::cin.tie(nullptr);
40
41     auto result = interpreter(tokens, input_file);
42
43     input_file.close();
44     return result;
45 }
```

```
1 | ./interpreter <code_file> <input_file>
```